# Fix C++26 by adding transposed special cases for P2642 layouts


| | |
|---|---|
| Document #: | P3222R1 |
| Date: | 2024-10-29 |
| Project: | Programming Language C++<br>LEWG |


| | |
|---|---|
| Reply-to: | Mark Hoemmen<br><mhoemmen@nvidia.com> |


# Contents

# § 1 Authors

— Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)

# § 2 Acknowledgements

Thanks to Nicolas Morales (Sandia National Laboratories) for review feedback.

# § 3 Revision history

— Revision 0 to be submitted for the post-Tokyo mailing before 2024/04/16

— Revision 1 to be submitted after LWG review on 2024-10-25

  — Improve Wording section by adding green and red text to express what has been changed. Make minor wording changes for consistency of word order with the Working Draft.

  — Suggestions from LWG pre-review and 2024-10-25 review:

    — Make Paragraph 4 consistent and more concise by using `a.stride(1)` and `a.stride(0)` instead of `a.mapping().stride(1)` and `a.stride(0)`, and `a.extents()` instead of `a.mapping.extents()`. This affects both existing and new wording.

    — Fix formatting of additions and deletions.

# § 4 Abstract

We propose to change the C++ Working Draft for C++26 so that `linalg::transposed` includes special cases for `layout_left_padded` and `layout_right_padded`. These are the two mdspan layouts proposed by P2642R6, which was voted into the C++ Working Draft at the Tokyo 2024 WG21 meeting. This change will make it easier for `linalg` implementations to optimize for these two layouts by dispatching to an existing optimized C or Fortran BLAS (Basic Linear Algebra Subroutines). Delaying this until after C++26 would be a breaking change.

# § 5 What the C++ Working Draft currently does

## § 5.1 What is `linalg::transposed`?

WG21 voted P1673R13 into the C++ Working Draft at the Kona 2023 WG21 meeting. P1673 introduces the `linalg::transposed` function, which takes a rank-2 `mdspan` and returns a read-only `mdspan` representing the transpose of its input. The *transpose* of a rank-2 mdspan `A` is a rank-2 mdspan `AT` such that `A[i, j]` refers to the same element as `AT[j, i]` for all `i, j` in the domain of `A`. Transposing a matrix "flips" it over its diagonal. The *diagonal* of a rank-2 mdspan `A` is the set of all elements `A[i, i]` where `i, i` is in the domain of `A`. A key feature of `transposed` is that it represents a read-only "transpose view" of the data, without copying or moving elements of the matrix.

## § 5.2 Special cases

The `transposed` function currently has "special cases" for three layouts: `layout_left`, `layout_right`, and `layout_stride`. For these three layouts, `linalg::transposed` works by changing the return type's layout and/or layout mapping in a way that reverses the extents and strides. For `layout_left`, "reversing the strides" means `layout_right`, and vice versa. Here are two examples.

1. The transpose layout mapping of `layout_left::mapping<extents<int, 3, 4>>{}` is `layout_right::mapping<extents<int, 4, 3>>{}`.

2. The transpose layout mapping of `layout_right::mapping<extents<int, 3, 4>>{}` is `layout_left::mapping<extents<int, 4, 3>>{}`.

For both `layout_left` and `layout_right`, the mapping does not store the strides; they are computed from the extents. For `layout_stride`, the mapping actually stores the strides and its constructor takes them as a `std::array`, so "reversing the strides" means passing in a reverse-order `array` of the input strides. For example, the transpose layout of

```
auto m = layout_stride::mapping<extents<int, 3, 4>>{
  extents<int, 3, 4>{}, array{2, 6}};
```

is

```
auto mt = layout_stride::mapping<extents<int, 4, 3>>{
  extents<int, 4, 3>{}, array{6, 2}};
```

The transpose layouts described above reflect the *current* behavior in the C++ Working Draft.

## § 5.3 Fall-back case

The *current* behavior in the C++ Working Draft is that for any layout which is not one of the three cases listed above (`layout_left`, `layout_right`, or `layout_stride`), `transposed` resorts to a "fall-back." That is, it wraps the original layout `Layout`'s mapping in a nested layout mapping `layout_transpose<Layout>::mapping`. That nested mapping's `operator()` invokes the original layout with indices reversed.

## § 5.4 Why does `transposed` need special cases?

The fall-back case correctly represents the transpose of an mdspan with any layout. If so, why do we need special cases? Why doesn't `transposed` just always use `layout_transpose`?

The design intent of P1673 is that implementations can dispatch to an existing optimized C or Fortran BLAS if the caller's `mdspan` arguments satisfy the right conditions. If the arguments do *not* satisfy these conditions, the implementation may dispatch to possibly unoptimized "generic" code. As P1673 explains, failure to dispatch to an optimized library may result in invocation of an asymptotically slower algorithm, and/or may fail to take advantage of any acceleration hardware. Some of these conditions depend only on the argument types and thus can always be checked at compile time, while other conditions depend on possibly run-time properties of the objects.

One of those conditions is that the layout mappings satisfy the following three properties.

1. They are all unique (`is_unique()` is `true`).

2. They are all strided (`is_strided()` is `true`).

3. At least one of their strides equals to one.

If all three are true, we say that the layout mapping is *BLAS-compatible*. For all `layout_left` and `layout_right` mappings, these are known at compile time always to be true. For `layout_stride`, testing the third property requires a possibly run-time check.

Knowing at compile time whether a layout mapping is BLAS-compatible makes it a zero-cost abstraction for the implementation to dispatch to the BLAS based on that mapping. As we will see below, both `layout_left_padded` and `layout_right_padded` are also known at compile time always to be BLAS-compatible. However, `transposed` does not have special cases for these two layouts.

## § 5.5 What if `transposed` had no special cases at all?

Suppose that `transposed` had no special cases for input layouts. Implementations of P1673's algorithms could still optimize by adding their own special cases for specific input layouts, such as `layout_transpose<layout_left>` and `layout_transpose<layout_right>`. This would not prevent implementations from dispatching to the BLAS. However, it would complicate the implementation and possibly add compile-time cost by introducing more internal overloads and/or specializations. Every algorithm would need to check for twice as many layout special cases: the BLAS-compatible layout, and `layout_transpose` of that layout. The code that dispatches to the BLAS would need to do the same thing that `transposed` does for its special cases, namely reverse the extents and strides in the transposed case. Furthermore, users may want to use `transposed` with their own P1673-like linear algebra algorithms. Such users would generally expect `transposed` to optimize for the common case of known strided layouts. Without that optimization, they may end up implementing their own `transposed`

functions. The proliferation of incompatible `transposed` functions would hinder interoperability of libraries.

## § 5.6 `layout_left_padded` and `layout_right_padded`

WG21 voted P2642R6 into the C++ Working Draft at the Tokyo 2024 WG21 meeting. P2642R6 adds two layouts, `layout_left_padded` and `layout_right_padded`. The data layouts described by these two class templates are exactly the two layouts understood by the C BLAS (Basic Linear Algebra Subroutines), as explained in P1673 and P1674. These layouts have one stride (the leftmost resp. rightmost) that is known at compile time to be one, and one stride (the next leftmost resp. rightmost) that the user provides either at compile time or run time. The remaining strides are computed from these and the extents, as if with `layout_left` resp. `layout_right` where the user-provided stride represents a possibly larger extent.

These two layouts are exactly the layouts supported by the BLAS. The BLAS calls the one user-provided stride the matrix's "leading dimension." (The name hints at the reason for these layouts, namely that they represent the layout of a submatrix of contiguous rows and columns of a possibly larger matrix, whose dimension is the user-provided stride.) BLAS implementations can optimize transpose of input matrices in these two layouts without copying data, just by reversing extents and retaining the one input stride. Furthermore, it is known at compile time that any mapping of these two layouts is BLAS-compatible. Therefore, it's reasonable to expect P1673 implementations to optimize for `layout_left_padded` and `layout_right_padded`. The way to do that would be for `transposed` of a `layout_left_padded<PaddingValue>` `mdspan` to return a `layout_right_padded<PaddingValue>` `mdspan` with extents swapped and the one "padding stride" copied over, and vice versa for `transposed` of a `layout_right_padded<PaddingValue>` `mdspan`. However, the C++ Working Draft currently handles those two layouts with the "fall-back" `layout_transpose` case.

## § 5.7 P1673 originally included this optimization

Earlier versions of P1673 defined two `mdspan` layouts, `layout_blas_general<column_major_t>` and `layout_blas_general<row_major_t>`. P1673's `transposed` function originally included special cases for those two layouts, as one can see in P1673R9's [linalg.transp.transposed]. Version R10 of P1673 moved those layouts to P2642 and renamed them `layout_left_padded` and `layout_right_padded`, respectively. P167310 removed these special cases from `transposed` so that P2642 and P1673 could make progress separately. However, P1673's authors always intended to optimize `transposed` for those layouts. WG21 voted P2642R6 into the C++ Working Draft at the Tokyo 2024 WG21 meeting, so now it's possible to carry out that intent.

# § 6 Proposed changes

We propose to add those two special cases. The result of `transposed` on a `layout_left_padded<PaddingValue>` `mdspan` will be a `layout_right_padded<PaddingValue>` `mdspan` with extents swapped and the one "padding stride" copied over. Likewise, the result of `transposed` on a `layout_right_padded<PaddingValue>` `mdspan` will be a `layout_left_padded<PaddingValue>` `mdspan` with extents swapped and the one "padding stride" copied over.

## § 6.1 Before and after example

The following example shows how this proposal would change the return type of `transposed`.

```
        // optimized overload
        extern void some_algorithm(
          mdspan<const float, dextents<size_t, 2>,
layout_right_padded<dynamic_extent>> A_T,
          mdspan<const float, dextents<size_t, 2>,
layout_left_padded<dynamic_extent>> B,
          mdspan<float, dextents<size_t, 2>,
layout_left_padded<dynamic_extent>> C);

        template<class GenericFallBackLayout>
        void some_algorithm(
          mdspan<const float, dextents<size_t, 2>, GenericFallBackLayout>
A_T,
          mdspan<const float, dextents<size_t, 2>,
layout_left_padded<dynamic_extent>> B,
          mdspan<float, dextents<size_t, 2>,
layout_left_padded<dynamic_extent>> C)
        {
          // ... slow generic code ...
        }

        void some_function(size_t N) {
          vector<float> A_storage(4 * N * N);
          vector<float> B_storage(N * N);
          vector<float> C_storage(N * N);

          mdspan A_parent{A_storage.data(), extents{2 * N, 2 * N}
          mdspan B{B_storage.data(), extents{N, N}};
          mdspan C{C_storage.data(), extents{N, N}};

          // ... fill A_parent and B with useful values ...

          // A views the upper left N x N submatrix of its "parent."
          mdspan A = submdspan(A_parent, tuple{0, N}, tuple{0, N});

          // Approval of P2642R6 added this to the C++ Working Draft.
          static_assert(is_same_v<
            decltype(A)::layout_type,
            layout_left_padded<dynamic_extent>>);
          static_assert(A.stride(0) == 1); // compile-time value
```

```
          assert(A.stride(1) == A_parent.stride(1)); // possibly run-time
value

          mdspan A_T = linalg::transposed(A);
          some_algorithm(A_T, B, C);
        }
```

### § 6.1.1 Before this proposal

1. `decltype(A_T)::layout_type` is `layout_transposed<layout_left_padded<dynamic_extent>>`.

2. Generic overload of `some_algorithm` is called.

### § 6.1.2 After this proposal

1. `decltype(A_T)::layout_type` is `layout_right_padded<dynamic_extent>`.

2. The statement `static_assert(A_T.stride(1) == 1);` is well formed.

3. Optimized overload of `some_algorithm` is called.

## § 6.2 Delaying until after C++26 would be a breaking change

The type of the layout of the `mdspan` returned by `transposed` is observable and is specified in the current wording. Therefore, delaying this change until after C++26 would be a breaking change.

## § 6.3 What happens if we don't do this?

We have already discussed above how the lack of special cases for `transposed` would affect use of `transposed` with user's custom P1673-like algorithms, and possibly hinder interoperability of different users' libraries. That leaves the effects of not having this proposal on P1673 implementations themselves.

The first and worst possibility is that implementations might not not optimize for `layout_left_padded` or `layout_right_padded` at all. That would be unfortunate, because those are exactly the layouts that the BLAS supports and has supported since the 1980's. This would hinder adoption of P1673 algorithms.

The second possibility is that implementations may nevertheless still dispatch to the BLAS for any BLAS-compatible layout. This works for user-defined layouts as well as the Standard layouts. `layout_transpose::mapping` preserves the uniqueness and stridedness of its nested layout mapping, and even reverses the strides if the nested layout mapping is strided. However, without this proposal, implementations would still need to check the actual stride values at run time, even though for `layout_left_padded` and

`layout_right_padded`, it's known at compile time that at least one of the strides is one. The run-time check would add overhead and complicate the implementation.

The third possibility is that implementations might add special cases for `layout_transpose<layout_left_padded<PaddingValue>>` and `layout_transpose<layout_right_padded<PaddingValue>>` in the algorithms, rather than in `transposed`. However, as discussed above in the section "What if `transposed` had no special cases?", this would complicate the implementation and possibly add compile-time cost.

The fourth possibility is that the implementation may simply not optimize the transposed case for any layouts. This would be unfortunate, as the BLAS itself favors transposes in some cases. For example, implementations of matrix-matrix multiply for general dense matrices (GEMM) have simpler code and may perform better if exactly one of the two input matrices is transposed. Under these so-called "NT" and "TN" cases, typical optimized implementations end up reading the matrices as if they have the same memory layout.

A valid P1673 implementation might not dispatch to the BLAS for all BLAS-compatible layouts. Instead, it might only do so for the four Standard layouts which are known to be BLAS compatible at compile time: `layout_left`, `layout_right`, `layout_left_padded`, and `layout_right_padded`. This approach has three advantages.

1. It minimizes run-time overhead by not calling `is_unique`, `is_strided`, or `stride`.

2. It can use function overloading on specific layout types to dispatch to the BLAS, instead of generic constraint checks (like a constraint that `is_always_strided()` is `true`) that may increase compilation cost.

3. It avoids the risk that user-defined layouts incorrectly define their stridedness or uniqueness. Users who copy-paste an existing layout to write their own might forget to change `is_strided()` or `is_unique()`.

However, without this proposal, such an implementation would need to resort to either the third or fourth possibility above.

# § 7 Optional design alternative: `transposed_mapping` customization point

In the previous section, we mentioned that users may want to use `transposed` with their own P1673-like linear algebra algorithms. A reviewer suggested that we make it possible for users to optimize `transposed` for their user-defined layouts, by adding a `transposed_mapping` customization point. This would be analogous to the `submdspan_mapping` customization point that approval of P2630R4 (`submdspan`) added to the C++ Working Draft. The new `transposed_mapping` customization point would take an input layout mapping and return the layout mapping that the transpose of an `mdspan` with

the input mapping would have. If no customization exists for a given layout mapping, `transposed` would default to using `layout_transpose`, as before.

This design would have the following advantages.

1. It would be easier to specify the wording of `transposed`. Instead of its current list of special cases, it would look more like the `submdspan` wording that puts all the layout-specific behavior in the customization point.

2. Implementations that provide implementation-specific layouts could optimize `transposed` for those layouts.

3. Users could use `transposed` with their custom P1673-like algorithms and implement optimizations for their user-defined layouts.

Here are some reasons why WG21 might *not* want to do this.

1. It would reserve yet another customization point name.

2. It would not change the set of BLAS-compatible layouts, and would not change the ability of P1673 implementations to dispatch to the BLAS for any BLAS-compatible layout.

3. `submdspan_mapping` enables functionality – the ability to slice `mdspan` with user-defined layouts. In contrast, `transposed_mapping` would only enable (or simplify) optimizations for one of many legal ways to implement the Standard.

4. It makes less sense for `transposed_mapping` to be a hidden friend than it does for `submdspan_mapping` to be a hidden friend. However, defining `transposed_mapping` as a nonmember function without using the hidden friends technique would make `transposed_mapping` vulnerable to implicit conversions. This could make `transposed`'s calls to the customization point ambiguous.

5. LEWG has already seen the proposed design over several reviews (the last being the 2022/07/05 telecon review of P1673R9), but has not yet had a chance to review this alternative customization point design.

Regarding (4), the C++ Working Draft defines `submdspan_mapping` customizations for Standard layout mappings as "hidden friends." The hidden friends technique protects use of the customization from possible ambiguities due to implicit conversions. This matters because layout mappings have many implicit conversions. These conversions help make `mdspan`-based interfaces more usable. Slicing is closely enough related to a layout mapping's behavior that it makes sense to put the slicing customization in the layout mapping. However, it makes less sense to make `transposed_mapping` a hidden friend of the mapping, because transposition only works for rank-2 mappings, and transposition is specific to linear algebra and related computations.

We think the disadvantages of a customization point outweigh the advantages. For example, we do not recommend adding `transposed_mapping` as a hidden friend of all the Standard layout mappings. We also would not want to force users to remember to protect their customizations from the possibility of ambiguous overloads. As a result, we do not provide wording for this alternative design. Nevertheless, we would like LEWG to poll this option.

# § 8 Implementation

This proposal is implemented as PR 268 in the reference `mdspan` implementation.

# § 9 Wording for the main proposal (not the alternative)

> Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording.
>
> Make the following changes to the latest C++ Working Draft as of the time of writing. All wording is relative to the latest C++ Working Draft. Inserted text is green and removed text () is red and overstruck.
>
> In [version.syn], increase the value of the `__cpp_lib_linalg` macro by replacing YYYMML below with the integer literal encoding the appropriate year (YYYY) and month (MM).

```
#define __cpp_lib_linalg YYYYMML // also in <linalg>
```

> Change [linalg.transp.transposed] paragraph 3 (“Let `ReturnExtents` be …”) by inserting two subparagraphs (as shown below) after subparagraph 3.2 (“otherwise, `layout_left` …”) and before current subparagraph 3.3 (“otherwise, `layout_stride` …”, to be renumbered to paragraph 3.5), and renumbering subparagraphs and subsubparagraphs within paragraph 3 thereafter.

3 Let `ReturnExtents` be *`transpose-extents-t`*`<Extents>`. Let `R` be `mdspan<ElementType, ReturnExtents, ReturnLayout, Accessor>`, where `ReturnLayout` is:

(3.1) `layout_right` if `Layout` is `layout_left`;

(3.2) otherwise, `layout_left` if `Layout` is `layout_right`;

(3.3) otherwise, `layout_right_padded<PaddingValue>` if `Layout` is `layout_left_padded<PaddingValue>` for some value `PaddingValue` of type `size_t`;

(3.4) otherwise, `layout_left_padded<PaddingValue>` if `Layout` is `layout_right_padded<PaddingValue>` for some value `PaddingValue` of type `size_t`;

(3.5) otherwise, `layout_stride` if `Layout` is `layout_stride`;

Change [linalg.transp.transposed] paragraph 4 (*Returns* clause of `transposed`) by inserting two subparagraphs (as shown below) after subparagraph 4.1 (for `Layout` being `layout_left`, `layout_right`, or a specialization of `layout_blas_packed`) and before current subparagraph 4.2 (for `Layout` being `layout_stride`, to be renumbered to subparagraph 4.4), and renumbering subparagraphs within paragraph 4 thereafter.

4 *Returns*: With `ReturnMapping` being the type `typename ReturnLayout::template mapping<ReturnExtents>`:

(4.1) if `Layout` is `layout_left`, `layout_right`, or a specialization of `layout_blas_packed`, `R(a.data_handle(), ReturnMapping(`*`transpose-extents`*`(a.~~mapping().~~extents())), a.accessor())`

(4.2) otherwise, if `Layout` is `layout_left_padded<PaddingValue>` for some value `PaddingValue` of type `size_t`, `R(a.data_handle(), ReturnMapping(`*`transpose-extents`*`(a.extents()), a.stride(1)), a.accessor())`

(4.3) otherwise, if `Layout` is `layout_right_padded<PaddingValue>` for some value `PaddingValue` of type `size_t`, `R(a.data_handle(), ReturnMapping(`*`transpose-extents`*`(a.extents()), a.stride(0)), a.accessor())`

(4.4) otherwise, if `Layout` is `layout_stride`, `R(a.data_handle(), ReturnMapping(`*`transpose-extents`*`(a.~~mapping().~~extents()), array{a.~~mapping().~~stride(1), a.~~mapping().~~stride(0)}), a.accessor())`